



DEPARTMENT OF
COMPUTER SCIENCE



INTRODUCTION TO PROBLEM SOLVING II

DR JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Describe how to carry out a solution plan to solve problems
- Apply concepts such as coding, debugging and program documentation
- Explain why we need to evaluate and refactor a solution



RECAP: FOUR STEPS TO PROBLEM-SOLVING

Understand the
problem

Create a solution
step-by-step
plan

Carry out the
solution plan

Evaluate and
refactor the
solution



CARRYING OUT A SOLUTION PLAN

After understanding the problem and creating a solution plan, the next step is to solve the problem. We need to tell the computer to solve the problem in a language it understands using computer programs.



CARRYING OUT A SOLUTION PLAN

Computer programs are also known as codes.

A very useful approach to adopt is to use the pseudocode as a guide to write out the program codes.



CARRYING OUT A SOLUTION PLAN

What if the entire problem can not be solved at once?

- An approach is to break down the entire problem into smaller simpler pieces and solve each part separately.
- It is better to focus on a simple solution as it is more likely to program a simpler solution correctly.



CARRYING OUT A SOLUTION PLAN

What if some parts of the problem can not be solved?

- In this case, we can ignore the difficult parts and focus on the easier aspects first.
- When all the easier parts are solved, it might then be possible to solve the harder parts.



WHAT DOES CARRYING OUT THE SOLUTION PLAN ENTAIL?

Carrying out the solution plan involves;

- Coding
- Compilation and Execution
- Debugging and Testing
- Program Documentation



CODING

This refers to the process of writing the actual program that solves the problem.

```
<div class="container">
  <div class="row">
    <div class="col-md-6 col-lg-8"> <!-- BEGIN NAVIGATION
      <nav id="nav" role="navigation">
        <ul>
          <li><a href="index.html">Home</a></li>
          <li><a href="home-events.html">Home Events</a></li>
          <li><a href="multi-col-menu.html">Multiple Column Men
          <li class="has-children"> <a href="#" class="current">
            <ul>
              <li><a href="tall-button-header.html">Tall But
              <li><a href="image-logo.html">Image Logo</a></li>
              <li class="active"><a href="tall-logo.html">Ta
            </ul>
          </li>
          <li class="has-children"> <a href="#">Carousels</a>
            <ul>
              <li><a href="variable-width-slider.html">Variab
              <li><a href="variable-width-slider.html">Testimonl
```



CODING

A coded program is most popularly referred to as a source code.

The coding process can be done in any high level or low level programming language.



CODING

High level languages are written in forms that humans can easily understand. Examples are C, C# and Java.

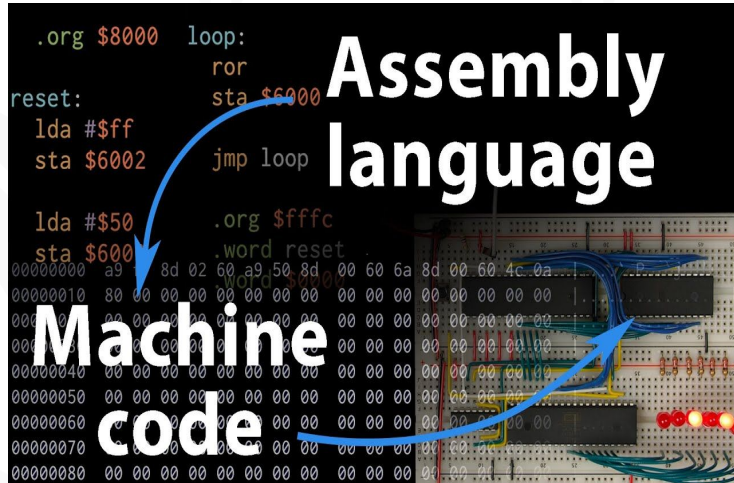
```
import java.io.*;
import java.util.Date;

public class SaveDate {

    public static void main(String args[]) throws Exception {
        FileOutputStream fos = new FileOutputStream("test.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        Date date = new Date();
        oos.writeObject(date);
        oos.flush();
        oos.close();
        fos.close();
    }
}
```



Low level languages are written in forms that the computer can easily understand. Examples are assembly language and machine code.





CODING

The computer only understands machine codes, but they are very difficult to write and troubleshoot because even small programs can have thousands of 0s and 1s.



Machine Code

```
10011101000110100000
01100011010001110110
10000010111101101110
11110110001011011000
10000010011100011011
10010011000111000000
```




CODING: AN EXAMPLE

The example below shows how pseudocodes can be converted into codes;

Pseudocode	Programming Code
1 START	<code>void sum(a,b) {</code>
2 READ N1 and N2	<code>int N1 = a;</code>
3 $R = N1 + N2$	<code>int N2 = b;</code>
4 PRINT R	<code>int R = N1+N2;</code>
5 STOP	<code>printf(R);</code>
	<code>}</code>



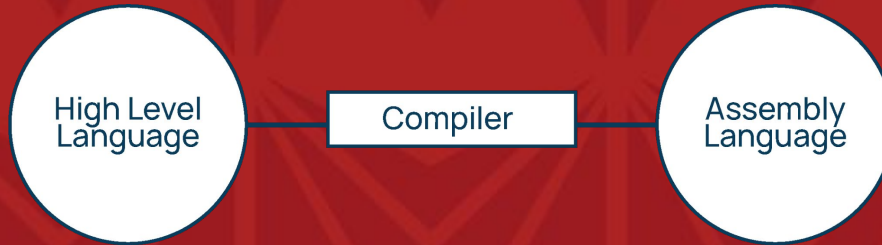
COMPILATION AND EXECUTION

Computer program are mostly written in either a high level language or with an assembly language. Interpreters are used to convert high level languages into machine code directly.



COMPILATION AND EXECUTION

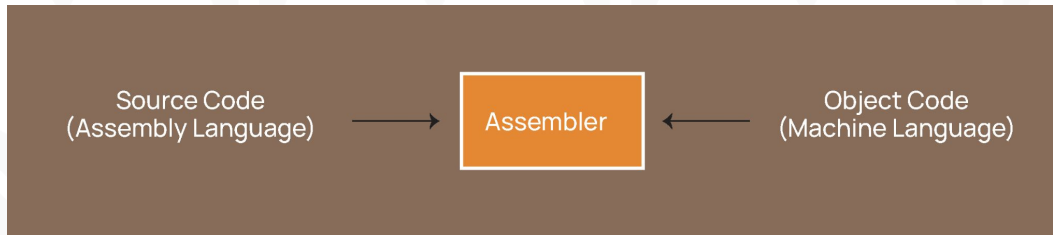
Compilers are used to convert high level languages into assembly languages.





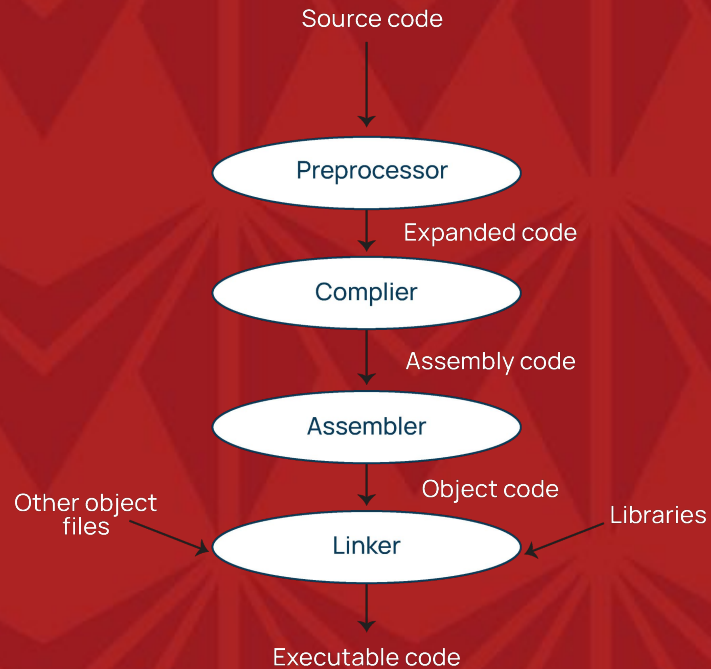
COMPILATION AND EXECUTION

Assemblers are used to convert assembly languages into machine codes.





COMPILATION AND EXECUTION PROCESS





DEBUGGING AND TESTING

When writing a computer program, errors such as syntax, logical and run time error can occur.



DEBUGGING AND TESTING

Debugging is a process that finds errors so that they can be removed to avoid program failure. Testing is a process that verifies if the computer program works as expected.



PROGRAM DOCUMENTATION

Program documentation is the process of collecting information about the program. The process starts from the problem analysis stage and continues to the testing stage.



PROGRAM DOCUMENTATION

There are two types of documentation;

- User Documentation: This contains the information the end user needs for the installation and use of the program.
- Programmer Documentation: This contains all the technical details so that the program can be upgraded and maintained effectively.



EVALUATING AND REFACTORING THE SOLUTION

Once a solution is working, the solution can be evaluated to see if it can be refactored.

Refactoring does not change the function of the solution, it is intended to make the solution more efficient while improving its structure and design.

Refactoring the solution will most likely result into a more concise and efficient solution.



EVALUATING AND REFACTORING THE SOLUTION

Questions to ask while trying to improve the solution include;

- Can we derive the result differently?
- Can we improve the performance of the solution?
- Can we think of other ways to refactor?
- How have other people solved the problem?



EVALUATING AND REFACTORING THE SOLUTION: AN EXAMPLE

The example below shows how a solution can be refactored for efficiency

Old Solution	New Solution
<pre>void sum(a,b) { int N1 = a; int N2 = b; int R = N1+N2; printf(R); }</pre>	<pre>void sum(a,b) { printf(a+b); }</pre>



FURTHER READING RESOURCES

- Bransford, J. D., & Stein, B. S. (1993). The IDEAL problem solver.
- Treffinger, D. J., Isaksen, S. G., & Stead-Dorval, K. B. (2006). Creative problem solving: An introduction. Prufrock Press Inc.
- Goodrich, M. T., & Tamassia, R. (2015). Algorithm design and applications (Vol. 363). Hoboken: Wiley.
- Khannur, A. (2014). Structured software testing: the discipline of discovering. Partridge Publishing India.
- Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
- Bailey, T. (2005). An introduction to the c programming language and software design. July-2005
-



REFERENCES

- *"Flying Magazine". Flying: 36–June 1963. ISSN 0015-4806.*
- *"Human Powered Flight Group Webpage". Royal Aeronautical Society. 2011. Retrieved 18 January 2021.*
- Bransford, J. D., & Stein, B. S. (1993). The IDEAL problem solver.
- Treffinger, D. J., Isaksen, S. G., & Stead-Dorval, K. B. (2006). Creative problem solving: An introduction. Prufrock Press Inc..



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU